

Weeks 5–6: Optimizers

Deep Learning – IITM BS

Notes with Intuition + Diagrams

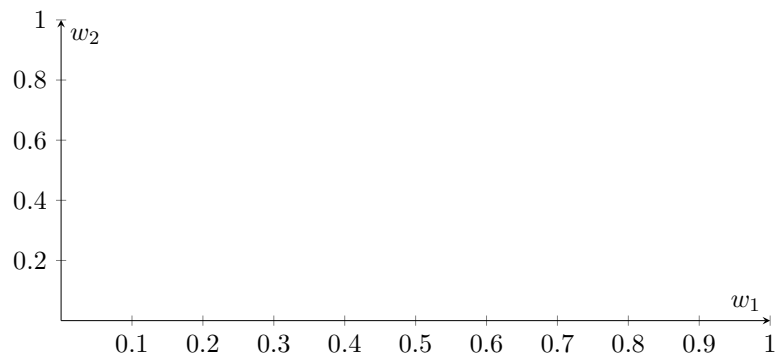
Contents

1	The Problem: Vanilla GD is Slow and Naive	2
2	Momentum-Based Gradient Descent	2
3	Nesterov Accelerated Gradient (NAG)	3
4	Batch, Stochastic, and Mini-Batch GD	3
5	Learning Rate Scheduling	3
6	Adaptive Learning Rate Methods	4
6.1	AdaGrad	4
6.2	RMSProp	4
6.3	AdaDelta	5
6.4	Adam (Adaptive Moments)	5
7	AdaMax and NAdam (Bonus Variants)	6
8	Practice Problems	7
9	Solutions	8

1 The Problem: Vanilla GD is Slow and Naive

Intuition:

Why do we need anything beyond plain gradient descent? Plain GD treats every direction and every step the same way—always the same step size η regardless of terrain. But real loss surfaces have valleys, plateaus, and saddle points. On a flat plateau, GD crawls forward pathetically slowly. On a steep valley, GD can overshoot and oscillate wildly. The entire family of optimizers exists to fix these two failure modes: **slow progress on flat regions** and **oscillation in steep/narrow valleys**.



Elongated Valley Problem

When the loss surface looks like an elongated bowl (very steep in one direction, very flat in another—high *curvature* in one dimension, low in the other), vanilla GD **zig-zags** back and forth across the steep direction while crawling painfully slowly along the flat direction.

2 Momentum-Based Gradient Descent

Intuition:

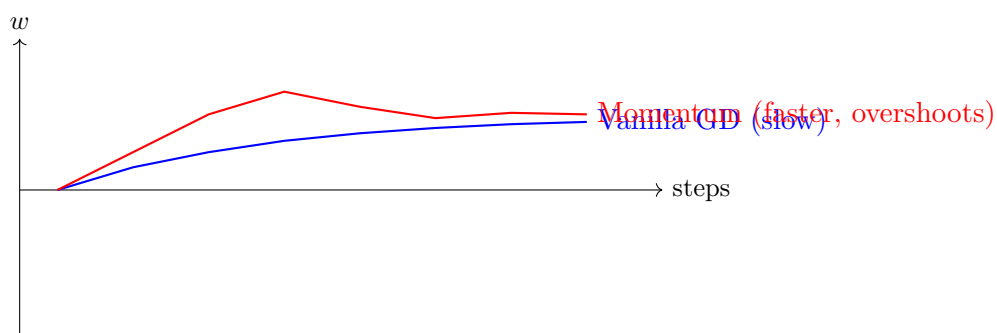
The rolling ball analogy Think of a ball rolling down a hill. If it's been rolling steadily in one direction, it builds up **momentum** and can roll straight through small bumps or flat patches without slowing down. Similarly, if gradient descent keeps getting told "go this way" over multiple steps, it should build confidence and take bigger strides in that consistent direction.

Momentum Update Rule

$$u_t = \beta u_{t-1} + \nabla w_t, \quad u_0 = \nabla w_0, \quad u_{-1} = 0$$

$$w_{t+1} = w_t - \eta u_t$$

$\beta \in [0, 1)$ typically ≈ 0.9 . Unrolling: $u_t = \sum_{\tau=0}^t \beta^{t-\tau} \nabla w_\tau$ — an **exponentially weighted average** of all past gradients, with recent gradients weighted more.



The Trade-off

Momentum accelerates convergence, especially on flat/gentle regions—but the accumulated momentum can cause it to **overshoot** the minimum, leading to oscillations before settling. It still converges faster overall than vanilla GD despite this.

3 Nesterov Accelerated Gradient (NAG)***Intuition:***

”Look before you leap” Momentum commits to a big jump first, then checks the gradient—like a person running full-speed and only looking where they’re going *after* taking the step. NAG is smarter: it takes a ”peek” step first (using the momentum direction alone), computes the gradient *from that peeked position*, and then makes the real update. This lookahead lets NAG ”see” if it’s about to overshoot and adjust course *before* committing, reducing oscillations.

NAG Update Rule

$$u_t = \beta u_{t-1} + \nabla(w_t - \beta u_{t-1})$$

$$w_{t+1} = w_t - \eta u_t$$

Compare to momentum: gradient is computed at the **look-ahead point** $w_t - \beta u_{t-1}$, not at w_t itself.

4 Batch, Stochastic, and Mini-Batch GD***Intuition:***

How much data to look at before updating? This is purely a question of *how much evidence* you gather before deciding to move. Look at the entire dataset (batch): very accurate direction but painfully slow per update. Look at just 1 point (stochastic): instant updates but very noisy, jittery direction. Mini-batch is the practical sweet spot: enough points to get a decent direction estimate, few enough to update quickly and often.

Algorithm	Points per Update	# Steps in 1 Epoch	Gradient Quality
Batch (Vanilla) GD	All N	1	Exact, but slow
Stochastic GD	1	N	Noisy, fast updates
Mini-batch GD	B	N/B	Balanced (used in practice)

5 Learning Rate Scheduling***Intuition:***

Why not just use one fixed learning rate forever? Early in training, you’re far from the minimum and want big, confident steps. Near the end, you’re close to the minimum and want small, careful steps to avoid overshooting and oscillating forever around the optimum. A fixed learning rate can’t satisfy both needs—so we *schedule* it to shrink over time.

Common Schedules

Step Decay: Halve η every K mini-batch **updates** (NOT epochs!)

Exponential Decay: $\eta = \eta_0 e^{-kt}$

1/t Decay: $\eta = \frac{\eta_0}{1 + kt}$

Cosine Annealing (warm restarts): $\eta_t = \eta_{\min} + \frac{\eta_{\max} - \eta_{\min}}{2} \left(1 + \cos \left(\pi \frac{t \% (T + 1)}{T} \right) \right)$

Exam Trap

Step decay tracks the number of **mini-batch parameter updates**, not epochs! Updates per epoch = N/B . Always compute total updates first, then check how many decay thresholds have been crossed.

6 Adaptive Learning Rate Methods

Intuition:

The core motivation Not all parameters deserve the same learning rate. A parameter connected to a rare feature (updates infrequently) should get bigger steps when it does update. A parameter connected to a very common feature (updates every step) should get smaller, more careful steps to avoid instability. Adaptive methods **personalize** the learning rate per parameter based on its update history.

6.1 AdaGrad

AdaGrad

$$v_t = v_{t-1} + (\nabla w_t)^2, \quad w_{t+1} = w_t - \frac{\eta}{\sqrt{v_t + \epsilon}} \nabla w_t$$

Intuition:

Why does this decay the LR for frequent parameters? v_t accumulates the *sum of squared gradients* for that parameter, forever growing. Parameters that get large/frequent gradients accumulate a large v_t , making $\eta/\sqrt{v_t + \epsilon}$ shrink a lot—slowing them down. **Problem:** since v_t only ever grows, eventually the effective learning rate for *every* parameter becomes tiny, and learning grinds to a halt—even if we're nowhere near done.

6.2 RMSProp

RMSProp – fixes AdaGrad's aggressive decay

$$v_t = \beta v_{t-1} + (1 - \beta)(\nabla w_t)^2, \quad w_{t+1} = w_t - \frac{\eta}{\sqrt{v_t + \epsilon}} \nabla w_t$$

Intuition:

The fix: forgetting old gradients Instead of an ever-growing sum, RMSProp uses an **exponentially decaying average** of squared gradients—old gradients gradually "fade out" of the memory. This means v_t can go up *or* down depending on recent gradient magnitudes, so learning never permanently grinds to a halt like in AdaGrad.

6.3 AdaDelta

AdaDelta – removes the need to set η_0

$$\begin{aligned} v_t &= \beta v_{t-1} + (1 - \beta)(\nabla w_t)^2 \\ \Delta w_t &= -\frac{\sqrt{u_{t-1} + \epsilon}}{\sqrt{v_t + \epsilon}} \nabla w_t, \quad w_{t+1} = w_t + \Delta w_t \\ u_t &= \beta u_{t-1} + (1 - \beta)(\Delta w_t)^2 \end{aligned}$$

Intuition:

Two histories running one step apart AdaDelta replaces the fixed η in the numerator with $\sqrt{u_{t-1}}$ —a running average of *past parameter update magnitudes* themselves. This is like saying: "my step size should match how big my recent steps have actually been," making the method self-calibrating without needing to hand-tune η_0 .

6.4 Adam (Adaptive Moments)

Intuition:

Combine the best of both worlds Adam = Momentum (tracks the *mean* of gradients, called the first moment m_t) + RMSProp (tracks the *variance*/magnitude of gradients, the second moment v_t). This gives Adam both a sense of "direction with confidence" (momentum) and "per-parameter step-size calibration" (adaptive LR).

Adam Update Rule – MUST MEMORIZE

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1 - \beta_1) \nabla w, & \hat{m}_t &= \frac{m_t}{1 - \beta_1^t} \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) (\nabla w_t)^2, & \hat{v}_t &= \frac{v_t}{1 - \beta_2^t} \\ w_{t+1} &= w_t - \frac{\eta}{\sqrt{\hat{v}_t + \epsilon}} \hat{m}_t \end{aligned}$$

Typical defaults: $\beta_1 = 0.9$, $\beta_2 = 0.999$.

Intuition:

Why the bias correction ($\hat{m}_t$, $\hat{v}_t$)? At the very start, $m_0 = v_0 = 0$, so early estimates of m_t, v_t are biased *too close to zero* (they haven't had time to "warm up" with enough gradient history). Dividing by $(1 - \beta^t)$ —which is small at first and approaches 1 as t grows—inflates the early estimates back to their proper unbiased scale. Without this correction, the first few updates would take unreasonably huge steps (since dividing by $\sqrt{\hat{v}_t}$ without correction would use an artificially tiny v_t).

Optimizer Cheat Table – EXAM CRITICAL

Optimizer	Uses Momentum?	Adaptive LR?
SGD	No	No
Momentum	Yes	No
NAG	Yes (with lookahead)	No
AdaGrad	No	Yes
RMSProp	No	Yes
AdaDelta	No	Yes
Adam	Yes	Yes

7 AdaMax and NAdam (Bonus Variants)**AdaMax**

Replaces the L^2 norm in Adam's denominator with an L^∞ (max) norm:

$$v_t = \max(\beta_2 v_{t-1}, |\nabla w_t|), \quad w_{t+1} = w_t - \frac{\eta}{v_t} \hat{m}_t$$

Notable: doesn't need bias correction for v_t (max-norm isn't biased toward zero the way a sum is).

NAdam

Incorporates the NAG "look-ahead" idea into Adam's momentum term, giving slightly faster, more responsive convergence than vanilla Adam in practice.

8 Practice Problems

Problem 1: Momentum Computation [3 marks]

$\beta = 0.9$, $\eta = 0.1$. Gradients: $\nabla w_0 = 2$, $\nabla w_1 = 1$. $u_{-1} = 0$, $w_0 = 5$.
Compute u_0, w_1, u_1, w_2 .

Problem 2: AdaGrad Computation [3 marks]

$\eta = 0.1$, $\epsilon = 10^{-8}$ (ignore in practice). Gradients over 2 steps: $\nabla w_0 = 4$, $\nabla w_1 = 2$. $v_{-1} = 0$, $w_0 = 10$.
Compute v_0, w_1, v_1, w_2 .

Problem 3: Step Decay [2 marks]

$\eta_0 = 0.5$, halved every 100 updates. Dataset $N = 20000$, batch size $B = 100$. What is η at the end of epoch 3?

Problem 4: Adam First Two Steps [3 marks]

$\beta_1 = 0.9$, $\beta_2 = 0.999$. Gradient at $t = 1$: $\nabla w = 2$. $m_0 = 0$, $v_0 = 0$.
Compute m_1 , $\hat{m}_1$, v_1 , $\hat{v}_1$. (Round to 4 decimals.)

Problem 5: Optimizer Identification [2 marks, MSQ]

Which optimizers use an adaptive (per-parameter) learning rate? Select all.
A. SGD B. Momentum C. AdaGrad D. RMSProp E. Adam

Problem 6: Mini-batch Steps [2 marks]

Dataset has 50,000 samples, batch size 250. How many parameter updates occur in 5 epochs?

9 Solutions

Problem 1: $u_0 = 0.9(0) + 2 = 2$. $w_1 = 5 - 0.1(2) = 4.8$.

$u_1 = 0.9(2) + 1 = 2.8$. $w_2 = 4.8 - 0.1(2.8) = \mathbf{4.52}$.

Problem 2: $v_0 = 0 + 16 = 16$. $w_1 = 10 - \frac{0.1}{\sqrt{16}}(4) = 10 - \frac{0.1}{4}(4) = 10 - 0.1 = 9.9$.

$v_1 = 16 + 4 = 20$. $w_2 = 9.9 - \frac{0.1}{\sqrt{20}}(2) = 9.9 - \frac{0.1}{4.47}(2) \approx 9.9 - 0.0447 = \mathbf{9.855}$.

Problem 3: Updates/epoch = $20000/100 = 200$. 3 epochs = 600 updates. Halvings at 100,200,300,400,500,600 = 6 times. $\eta = 0.5 \times 0.5^6 = 0.5/64 \approx \mathbf{0.0078}$.

Problem 4: $m_1 = 0.9(0) + 0.1(2) = 0.2$. $\hat{m}_1 = 0.2/(1 - 0.9) = 0.2/0.1 = \mathbf{2.0}$.

$v_1 = 0.999(0) + 0.001(4) = 0.004$. $\hat{v}_1 = 0.004/(1 - 0.999) = 0.004/0.001 = \mathbf{4.0}$.

Problem 5: C, D, E. AdaGrad, RMSProp, Adam all adapt LR per parameter.

Problem 6: Updates/epoch = $50000/250 = 200$. 5 epochs = $200 \times 5 = \mathbf{1000}$ updates.